



CLIENT/SERVER NETWORK MODEL

DEBMALYA CHAKI

15600123014 B.tech,CSE,6th Sem



Introduction

The client/server model is a distributed application architecture that divides tasks between servers and clients. Servers are responsible for providing services, resources, or data, while clients send requests to access these services. This model forms the foundation of many modern networked systems, including web applications, email services, and file-sharing platforms.

In Java, the client/server model is implemented using sockets, which enable communication between devices over a network through the TCP/IP protocol. The `java.net` package provides essential classes such as `ServerSocket` and `Socket`, allowing developers to create reliable network-based applications by establishing connections between clients and servers.

Basic Concepts

Before developing a client/server model, it is important to understand some fundamental networking concepts. These core components form the foundation of communication between systems in a distributed environment.

2.1 Client

A client is a program or device that initiates communication with a server. It sends requests to the server in order to access services or resources and then receives responses.

For example, a web browser acts as a client when it requests a webpage from a web server.

2.2 Server

A server is a program or system that waits for incoming client requests. It processes these requests and sends back the appropriate responses.

For example, a web server responds to a browser by sending HTML content that displays a webpage.

2.3 Socket

A socket is an endpoint for communication between two machines over a network. It enables data exchange between a client and a server.

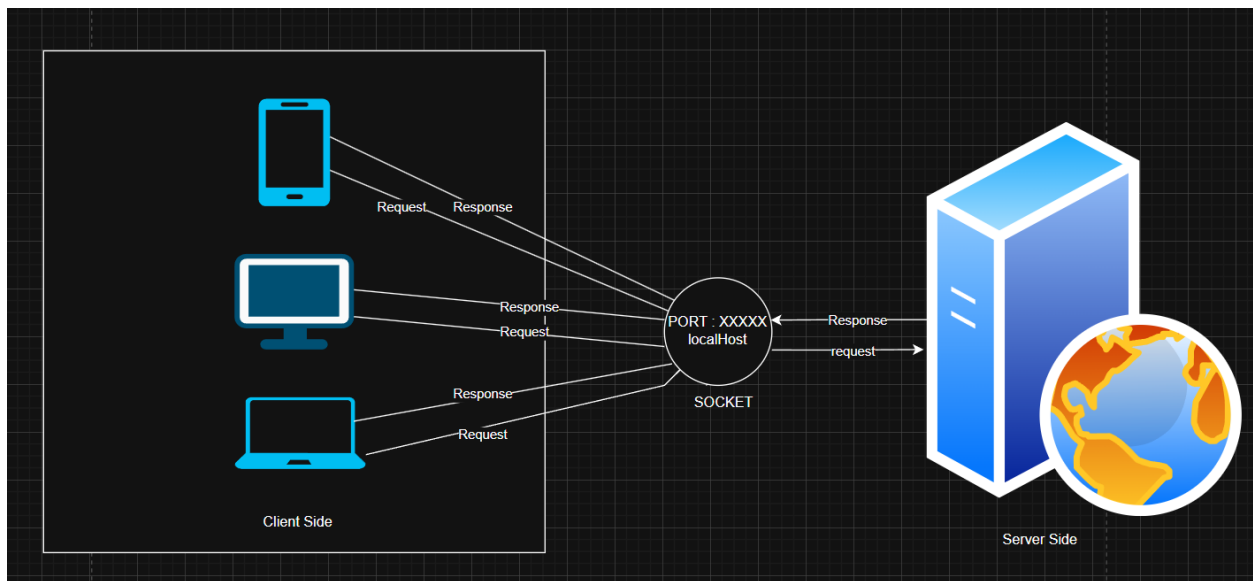
In Java, socket communication is supported through:

- `ServerSocket` – used on the server side to listen for client connections.
- `Socket` – used on the client side to connect to the server.

2.4 TCP/IP Communication

TCP/IP (Transmission Control Protocol/Internet Protocol) is a reliable, connection-oriented communication protocol. It ensures that data is delivered accurately, in sequence, and without loss during transmission between client and server.

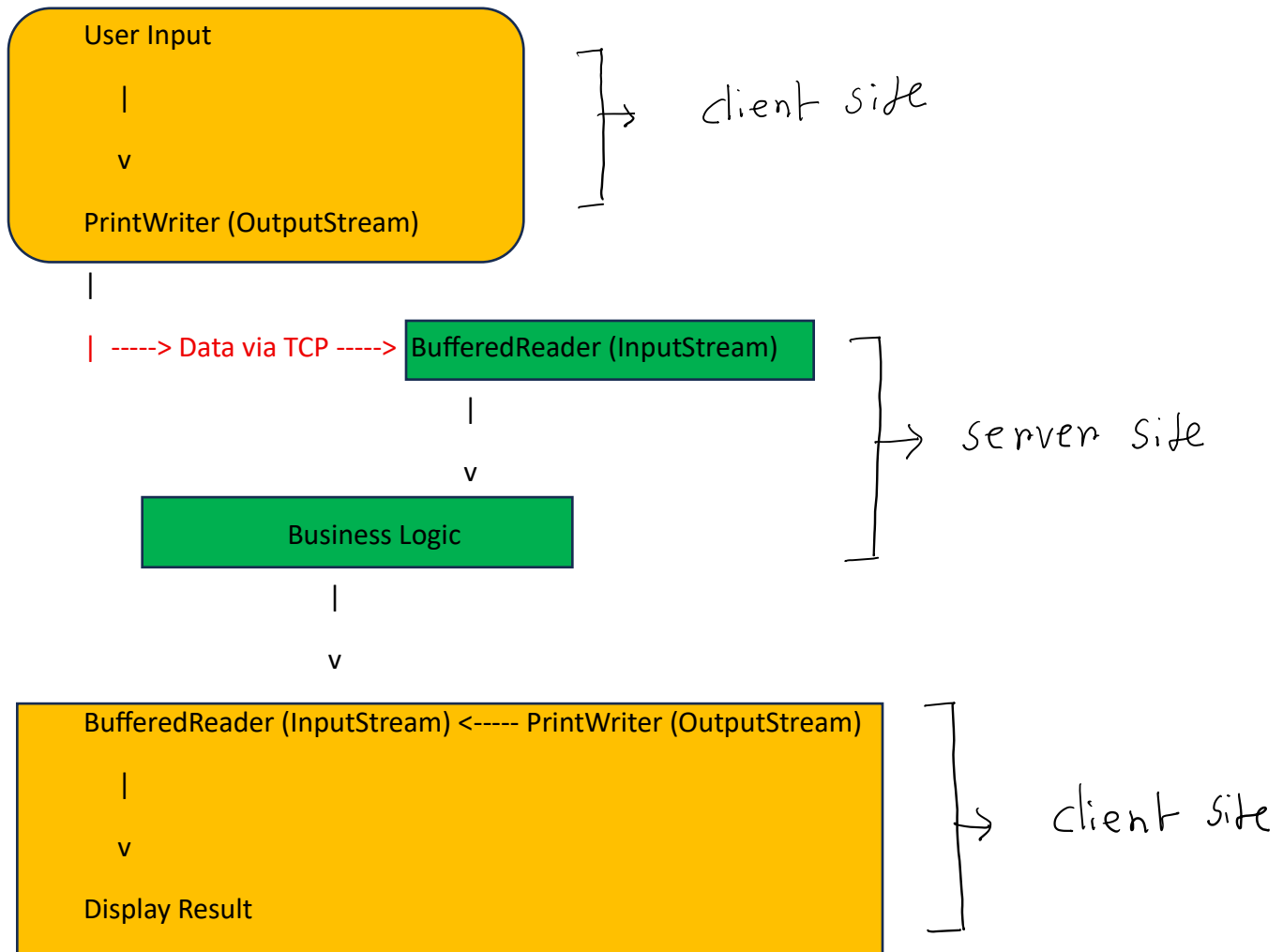
Client/Server Architecture



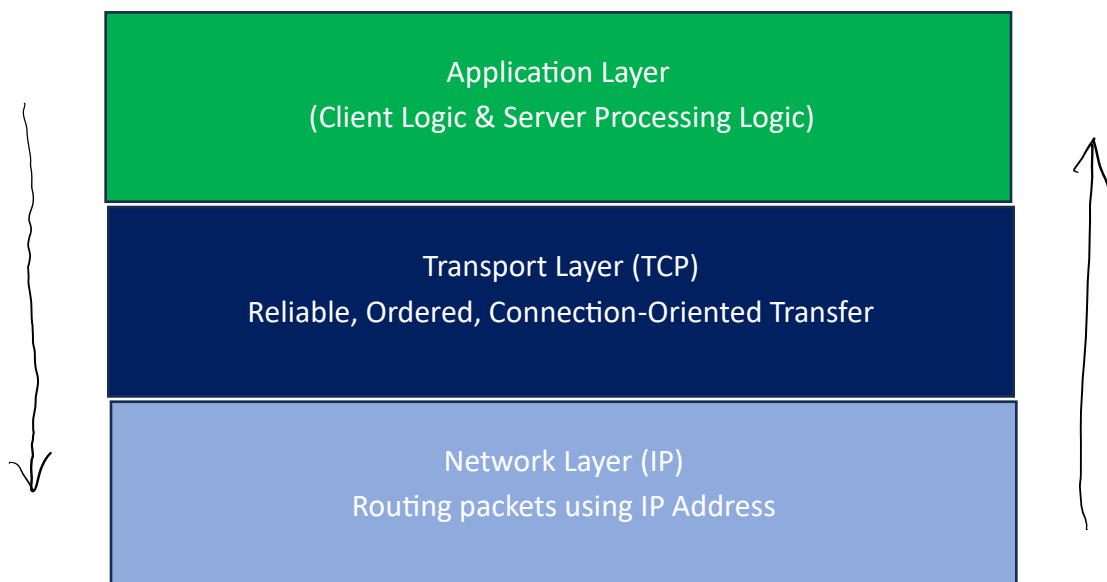
The **server** runs continuously, listening for incoming client connections.

The **client** connects, sends requests, and waits for responses.

Detailed Data Flow (Stream Level View)



System View (Layer Wise)



When program **writes data (Output / send)**: READ TOP → DOWN

When program **reads data (Input / receive)**: READ BOTTOM → UP

Server-Side Code Skeleton:

The server program is responsible for:

- Listening on a specific port
- Accepting client connections
- Receiving messages from the client
- Sending responses back
- Closing the connection properly

The implementation is explained part by part below.

Importing Required Packages :

The java.net package provides the networking classes required for communication over TCP/IP. In this program, it is used to create ServerSocket and Socket objects. The java.io package provides input and output stream classes such as BufferedReader, InputStreamReader, and PrintWriter, which are used for sending and receiving data over the network connection.

```
import java.io.*;
import java.net.*;
```

The execution of the program starts from the main method:

```
public class SimpleServer { new *

    public static void main(String[] args) throws Exception { new *
```

The main method serves as the entry point of the Java application. The keyword `static` allows the method to be executed without creating an object of the class. The declaration `throws Exception` ensures that any network-related exceptions can be handled properly without terminating the program abruptly.

Next, the server defines the port number on which it will listen for incoming client connections:

The port number uniquely identifies the service running on a system. In this case, port 5000 is used for communication. The client must connect using the same port number; otherwise, the connection will fail.

```
int port = 5000;
```

After defining the port number, the server creates a `ServerSocket` object:

```
try(ServerSocket serverSocket = new ServerSocket(port)){
```

The `ServerSocket` binds the server to port 5000 and places it in a listening state. At this stage, the server is ready to accept client requests but does not yet communicate with any specific client. The `try-with-resources` statement ensures that the socket will automatically close when execution completes.

Once the server starts successfully, a confirmation message is displayed:

```
System.out.println("Server Is Starting.....");
```

This message indicates that the server has initialized correctly and is waiting for a client to connect.

The next step is to accept a client connection:

```
// wait until a client can send a request  
  
Socket clientSocket = serverSocket.accept();
```

The `accept()` method blocks the execution of the program until a client attempts to connect. When a client sends a connection request, the TCP three-way handshake is performed automatically by the operating system. After a successful handshake, a new `Socket` object is created to represent the communication channel between the server and that specific client.

To identify the connected client, the server retrieves and prints the client's IP address:

```
// print IP address

System.out.println("Client Connected " + clientSocket.getInetAddress());
```

The `getInetAddress()` method returns the IP address of the connected client. This is useful for monitoring and logging purposes.

After establishing the connection, the server sets up the input stream to receive data from the client:

```
// Stream Setup
// Reader
BufferedReader bufferedReader = new BufferedReader(
    (new InputStreamReader(clientSocket.getInputStream())));
```

The `getInputStream()` method retrieves raw byte data from the client. Since communication involves text messages, `InputStreamReader` converts the byte stream into a character stream. The `BufferedReader` then reads the data efficiently line by line. This stream is responsible for receiving the client's request messages.

Similarly, the server prepares the output stream to send responses back to the client:

```
PrintWriter printWriter = new PrintWriter(clientSocket.getOutputStream(), autoFlush: true);
```

The `getOutputStream()` method sends data to the client. The `PrintWriter` allows the server to send formatted text messages easily. The parameter `true` enables

automatic flushing, which ensures that data is transmitted immediately rather than being stored in a buffer.

After setting up the streams, the server enters a loop to continuously receive and respond to client messages:

The `readLine()` method waits until the client sends a message. When a message is received, it is displayed

on the server console.

The server then sends a response back to the client using the `printWriter`. This loop continues as long as the client remains connected and sends messages. If the client disconnects, `readLine()` returns null, and the loop terminates.

```
String message;

while((message = bufferedReader.readLine()) != null) {

    System.out.println("Server Received Message: " + message);

    printWriter.println("Server : " + message);

}

clientSocket.close();
```

Closing the socket releases network resources and terminates the TCP connection gracefully. The TCP termination process ensures that all remaining data is transmitted before closing the connection.

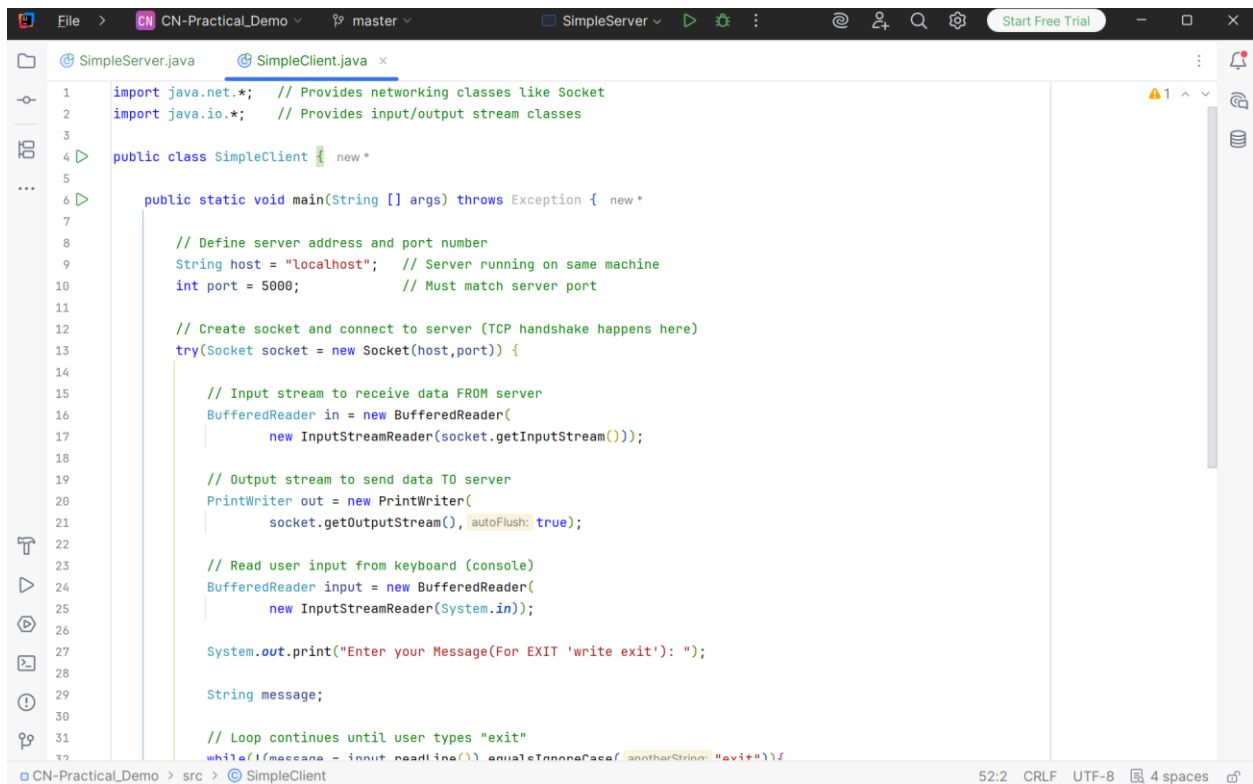
In case any network error occurs, such as the port already being in use, the program handles the exception:

```
    } // End of the try block  
    catch (IOException e){  
        System.err.println("Could not listen on port: "+port);  
    }  
}
```

This prevents the program from crashing and provides a meaningful error message to the user.

In conclusion, the server program follows a systematic process. It starts by binding to a port, waits for client connections, establishes communication using sockets, sets up input and output streams for data transfer, processes client messages, sends responses, and finally closes the connection. This structured approach ensures reliable communication in the client–server model using TCP/IP.

Client Side :



```
1 import java.net.*; // Provides networking classes like Socket
2 import java.io.*; // Provides input/output stream classes
3
4 public class SimpleClient { new *
5
6     public static void main(String [] args) throws Exception { new *
7
8         // Define server address and port number
9         String host = "localhost"; // Server running on same machine
10        int port = 5000; // Must match server port
11
12        // Create socket and connect to server (TCP handshake happens here)
13        try(Socket socket = new Socket(host,port)) {
14
15            // Input stream to receive data FROM server
16            BufferedReader in = new BufferedReader(
17                new InputStreamReader(socket.getInputStream()));
18
19            // Output stream to send data TO server
20            PrintWriter out = new PrintWriter(
21                socket.getOutputStream(), autoFlush: true);
22
23            // Read user input from keyboard (console)
24            BufferedReader input = new BufferedReader(
25                new InputStreamReader(System.in));
26
27            System.out.print("Enter your Message(For EXIT 'write exit'): ");
28
29            String message;
30
31            // Loop continues until user types "exit"
32            while(! (message = input.readLine()).equalsIgnoreCase( anotherString: "exit")){
33
34                out.println(message);
35                // Send message to server (Client → Server)
36
37                System.out.println("Server Says :"+in.readLine());
38                // Receive reply from server (Server → Client)
39            }
40
41            // When loop ends, socket automatically closes (try-with-resources)
42
43        } catch (UnknownHostException e) {
44            // Error if server host cannot be found
45            throw new RuntimeException(e);
46        } catch (IOException e) {
47            // Error during input/output communication
48            throw new RuntimeException(e);
49        }
50    }
51 }
52 }
```

the working of a simple client application using Java socket programming. The classes from java.net and java.io packages are imported to enable networking and input/output operations. The java.net package provides the Socket class to

establish a connection between the client and server, while the `java.io` package provides stream classes for sending and receiving data.

The execution of the program begins in the main method. First, the server address and port number are defined. The host is set to `"localhost"`, which means the server is running on the same machine. The port number is set to 5000, and it must match the port number used by the server application.

A socket object is then created using `new Socket(host, port)`. When this line executes, a TCP connection is established between the client and the server. This is where the TCP three-way handshake process takes place automatically. The program uses a try-with-resources block, which ensures that the socket is automatically closed once the communication ends.

After establishing the connection, input and output streams are created. A `BufferedReader` named `in` is used to receive data from the server through the socket's input stream. A `PrintWriter` named `out` is used to send data to the server through the socket's output stream. Another `BufferedReader` named `input` is created to read messages entered by the user from the keyboard.

The program then prompts the user to enter a message. A loop begins, which continues running until the user types `"exit"`. Inside the loop, the message entered by the user is sent to the server using `out.println(message)`. After sending the message, the client waits to receive a response from the server using `in.readLine()`, and the server's reply is displayed on the console.

When the user types `"exit"`, the loop terminates, and the socket connection is automatically closed because of the try-with-resources structure. If any error occurs, such as the server not being found or a communication failure, the program handles it using exception handling for `UnknownHostException` and `IOException`.

Overall, this client program establishes a TCP connection with the server, sends messages entered by the user, receives responses from the server, and terminates the connection when the user decides to exit.

Outputs :



The screenshot shows the IntelliJ IDEA Run console with two tabs: SimpleServer and SimpleClient. The SimpleServer tab is active, displaying the following output:

```
C:\Users\debma\jdk\ms-21.0.10\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.3\lib\idea_rt.jar=566...
Server Is Starting.....
Client Connected /127.0.0.1
Server Received Message: Hi
Server Received Message: Hello
Server Received Message: how are you
|
```

The status bar at the bottom indicates the file path is CN-Practical_Demo > src > SimpleClient, with encoding 52:2, CRLF, UTF-8, and 4 spaces.



The screenshot shows the IntelliJ IDEA Run console with two tabs: SimpleServer and SimpleClient. The SimpleClient tab is active, displaying the following output:

```
C:\Users\debma\jdk\ms-21.0.10\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.3\lib\idea_rt.jar=656...
Enter your Message(For EXIT 'write exit'): Hi
Server Says :Server :Hi
Hello
Server Says :Server :Hello
how are you
Server Says :Server :how are you
exit
Process finished with exit code 0
```

The status bar at the bottom indicates the file path is CN-Practical_Demo > src > SimpleClient, with encoding 52:2, CRLF, UTF-8, and 4 spaces.

THANK YOU